CogniStream Telemetry Pipeline

Week 4 Engineering Guide & Architecture Documentation

Project:	CogniStream Telemetry	**Stage:**	Week 4 Orchestration
Tech Stack:	Python, Polars, chDB, Docker, Airflow	**Status:**	Validated & Production Ready

1. Overview & Objectives

Week 4 transitions CogniStream from simple manual execution scripts to an automated, production-grade orchestration workflow managed by Apache Airflow. High-throughput telemetry data is generated, normalized, and persisted into columnar Parquet storage, queried seamlessly via an embedded ClickHouse (`chdb`) engine.

> **Core Architecture Target:**
> Mock Telemetry Stream → Polars Schema Normalization → ClickHouse Engine (`chdb`) Storage → Automated Metrics Computation

2. Technical Architecture & Component Roles

Component	Role & Responsibility in Ingestion Pipeline
Apache Airflow	Orchestrates scheduled DAG task execution, dependency tracking, retries, and failure alerts in Docker.
Polars Engine	Executes fast, vectorized schema alignment, timestamp parsing, and data cleaning on raw JSON streams.
chdb (ClickHouse)	Serves as an embedded OLAP engine running high-speed SQL queries directly on persistent Parquet datasets.
Docker Storage	Provides isolated container runtime environment with persistent data storage mapped to `/opt/airflow/chdb_data/`.

3. Airflow DAG Implementation (`cognistream_dag.py`)

The ingestion DAG runs on an hourly schedule with standard retry logic and sequential task dependencies.

FILE: /OPT/AIRFLOW/DAGS/COGNISTREAM_DAG.PY

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'cognistream',
    'depends_on_past': False,
    'start_date': datetime(2026, 1, 1),
    'email_on_failure': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=2),
}

with DAG(
    'cognistream_ingestion',
    default_args=default_args,
    description='End-to-end CogniStream Telemetry Ingestion Pipeline',
    schedule_interval='@hourly',
    catchup=False,
) as dag:

    generate_events = BashOperator(
        task_id='generate_mock_events',
        bash_command='python3 /opt/airflow/generate_mock_events.py',
    )

    normalize_data = BashOperator(
        task_id='normalize_events',
        bash_command='python3 /opt/airflow/normalize_events.py',
    )

    load_clickhouse = BashOperator(
        task_id='load_to_clickhouse',
        bash_command='python3 /opt/airflow/load_to_clickhouse.py',
    )

    run_metrics = BashOperator(
        task_id='base_metrics',
        bash_command='python3 /opt/airflow/base_metrics.py',
    )

    # Sequential Dependency Chain
    generate_events >> normalize_data >> load_clickhouse >> run_metrics
```

4. Operational & Verification Commands

Execute End-to-End Ingestion Pipeline:

```
docker exec -it cognistream-airflow bash -c "python3 /opt/airflow/generate_mock_events.py && python3 /opt/airflow/normalize_events.py && python3 /opt/airflow/load_to_clickhouse.py && python3 /opt/airflow/base_metrics.py"
```

Inspect Record Counts via chdb SQL:

```
docker exec -it cognistream-airflow python3 -c "import chdb; print(chdb.query("SELECT count(*) AS total_records FROM file('/opt/airflow/chdb_data/raw_events.parquet', Parquet)", 'Pretty'))"
```

Query Top Telemetry Events:

```
docker exec -it cognistream-airflow python3 -c "import chdb; print(chdb.query("SELECT * FROM file('/
opt/airflow/chdb_data/raw_events.parquet', Parquet) LIMIT 10", 'Pretty'))"
```

5. Mid-Project Review Checklist

- ✔ Standardized all root Python scripts in plain text, resolving prior RTF formatting syntax errors.
- ✔ Validated persistent Docker volume mounting at `/opt/airflow/chdb_data/` across container restarts.
- ✔ Standardized ClickHouse query syntax using `file('/path/file.parquet', Parquet)` table function.
- ✔ Configured and verified end-to-end Airflow orchestration DAG execution and reporting.